Issues found in `Complex_Time_QHO_Anharmonic.ipynb`

Ross Jenkinson

16 July 2026

Summary

I cross-checked the notebook against a deterministic (non-ML, non-MC) evaluation of the *identical* discretised path integral: same $N = 32$, same complex step $a_c = T_c/N$ with $T_c = 1 - 0.05i$, same action and normalisation, computed exactly by transfer matrix (iterated FFT quadrature, converged to $\gtrsim 7$ digits and validated at $\lambda = 0$ against the closed-form lattice Gaussian and the Mehler kernel). The overall framework (contour deformation, $\log W = iS + \log\det J - \log P$, variance loss) is sound, but there are two genuine bugs and one systematic effect. Together they plausibly account for the disagreement with my numbers.

1. Transpose bug in `compute_log_det_J` (definite bug)

The forward pass computes `x = z @ A.t() + b + g(z)`, i.e. $x = Az + b + g(z)$, so the exact Jacobian is

$$J \;=\; A + C\,D_2W_2D_1W_1, \qquad C = W_3^{(\mathrm{re})} + i\,W_3^{(\mathrm{im})}, \quad D_k = \mathrm{diag}(1 - a_k^2),$$

and by Sylvester's identity

$$\det J \;=\; \det A \cdot \det\bigl(I_r + D_2W_2D_1W_1\,A^{-1}C\bigr).$$

The code instead uses $A^{-\top}C$:

```
A_invT_C = torch.linalg.solve(A.t(), C)   # computes A^{-T} C  -- WRONG
A_inv_C  = torch.linalg.solve(A, C)       # correct for x = z @ A.t()
```

This is only consistent with a forward pass $x = A^\top z$ (as in the markdown), not with the code's $x = Az$; the two agree only for symmetric A, which does not hold after training. I verified numerically against finite differences: your formula is wrong at $\mathcal{O}(10^{-2})$ in $\log\det J$ for generic parameters, the corrected one is exact to machine precision. The bug biases the final estimator (and the training loss). One-character fix above, or change the forward pass to `z @ A`.

2. Contour orientation and homology are unconstrained (design flaw)

In your saved outputs (run with $\lambda = 0.5$, seeds 42/43), the two seeds return

$$K_{42} = +0.31706 - 0.29829i, \qquad K_{43} = -0.31708 + 0.29826i,$$

i.e. *exactly opposite signs*, both with loss $\sim 3 \times 10^{-4}$ and $\mathrm{ESS}/N = 0.9997$. The variance loss is blind to any constant phase in $\log W$, so it cannot distinguish a contour from its parity-reversed image; with $D = 31$ odd, $A \to -A$ flips $\det J$ and hence the sign of K while leaving the loss untouched. More generally nothing anchors the learned manifold to the homology class of $\mathbb{R}^D$ and nothing penalises folding ($\det J \to 0$), so the network can converge to a beautiful low-variance answer to a *different* integral. Suggestions: track the contour continuously from the near-identity initialisation, monitor $\min_z |\det J(z)|$ during training, and treat seed-to-seed scatter beyond the quoted error bars as an error signal, never something to average over.

3. Quantified bias and reference values

Exact values for your discretisation and for the continuum ($K = \sum_n |\psi_n(0)|^2 e^{-iE_n T_c}$, Fourier-spectral diagonalisation), all at $x_i = x_f = 0$, $T_c = 1 - 0.05i$, $V = \frac{1}{2}x^2 + \frac{\lambda}{4}x^4$:

λ	Exact lattice (N=32)	Continuum	Notebook ML
0	$0.3121159 - 0.3022604i$	$0.3121370 - 0.3022849i$	not run
0.5	$0.3057603 - 0.3077905i$	$0.3063511 - 0.3082963i$	$0.3170554 - 0.2982940i$
0.7	$0.2684413 - 0.2878695i$	$0.2684401 - 0.2897428i$	—
2.4	$-0.1109040 - 0.1529262i$	$-0.1118026 - 0.1577383i$	—

Two consequences. First, the $\lambda = 0.5$ ML result is 3.4% from the exact value of *its own* lattice integral, roughly 20× the nominal MC error, so the estimator is biased independently of discretisation. Second, lattice vs. continuum differs by 0.18% at $\lambda = 0.5$ and 2.5% at $\lambda = 2.4$: even a corrected ML run at fixed $N = 32$ will not match continuum benchmarks at strong coupling without an $N \to \infty$ extrapolation.

4. Minor issues

(i) The potential sum runs over interior slices only; harmless for $x_i = x_f = 0$, a latent bug otherwise (standard convention includes $V(x_0)$). (ii) `compute_hessian_A_init` builds the Hessian with the real step T/N instead of a_c (currently unused, but will matter if re-enabled). (iii) Everything is `complex64`; move the action and $\log W$ accumulation to `complex128`. (iv) The $\lambda = 0$ sanity check is skipped in the current configuration, and the saved outputs do not match the current source (λ list and seeds differ); please rerun clean. (v) Worth confirming we share the $\lambda x^4/4$ (not $\lambda x^4/4!$) convention.

Suggested validation order

Fix the transpose; rerun $\lambda = 0$ against the closed-form lattice Gaussian (not Mehler, and much tighter than the current 5% threshold); benchmark $\lambda > 0$ against the transfer-matrix values above at the same N; only then compare to continuum results with an $N \to \infty$ extrapolation. The transfer matrix is the decisive arbiter: it evaluates the identical integral with no ML and no Monte Carlo. Scripts: `exact_check.py`, `jacobian_check.py`.